甘肃专升本

理工类公共课计算机

C 语言基础知识总结

主要内容：1.C 数据类型

2.简单的算术运算和表达式

3.键盘输入和屏幕输出

4.选择控制结构

5.循环控制结构

甘肃专升本网 WWW.515148.COM

一、C 语言的数据类型

以下为基础知识点，在答题时常会遇到：

1.C 语言处理两种数据：常量和变量
2.常量：整形常量、实型常量、字符常量、字符串常量、枚举常量
3.整形常量：正整数、负整数、零　　eg: 10,-30,0
4.实型常量：实数、小数、浮点数　　eg: 3.14,-0.56,18.0
5.字符常量：字符型常量、用单引号括起来的任意字符　eg: 'x'　'X'　'?'　','　'9'
6.字符串常量：用双引号括起来的一个或多个字符　　eg: "x"　"Hello"　"Y78"
7.在 C 程序中，整型常量可以用十进制、八进制、十六进制形式表示，但编译器都会自动将其转换成二进制存储
8.整型常量的类型：长整型、短整型、有符号、无符号。
9.不同进制的整型常量的表示形式：
　　1）十进制：0~9 数字组合，可带正负号　　eg: 256,-128,0,+7
　　2）八进制：0 开头，后面跟 0~7 数字组合　　eg: 021,-017
　　3）十六进制：0x(0X)开头，后跟 0~9,a~f(A~F)数字组合　eg:0x12,-0x1F
10.不同类型的整形常量的表示形式：
　　1) 有符号整型常量：默认 int 型定义为有符号整数，无需 signed　　eg:10,-30,0
　　2) 无符号整形常量：常量值后跟 u 或 U,只能表示>=0 的数　　eg:30u,256U
　　3) 长整型常量：常量值后跟 l 或 L　　eg:-256l,1024L
　　4) 无符号长整型常量：常量值后跟 LU、Lu 、lU 、lu　　eg:30lu
11.C 程序中，实型常量两种表示形式：小数、指数
12.不同形式的实型常量的表示形式：
　　1) 十进制表示形式：数字和小数点组成，必须要有小数点，整数部分可省略
　　eg:0.123,　-12.35,　.98
　　2) 指数形式：e 或 E 代表以 10 为底的指数，　e 左边是数值部分(有效数字，可以是整数、小数，不能省略)，e 右边是指数部分(必须是整数形式)　　eg:3.45e-6
13.实型常量有单精度和双精度之分，无有符号和无符号之分
14.不同类型的实型常量表示形式：
　　1)单精度实型常量：常量值后跟 F 或 f　　eg:1.25F,1.25e-2f
　　2)双精度实型常量(double)：实型常量默认按双精度处理　　eg:0.123,-12.35,　.98
　　3)长双精度实型常量(long double)：常量值后跟 L 或 l　　eg:1.25L
15.变量：在程序执行过程中可以改变的量
16.变量在必须先定义后使用，定义时需要声明变量的类型和变量名，一般形式：类型关键字　变量名;
17.关键字是 C 语言预先规定具有特殊意义的单词，类型关键字用于声明变量的类型
18.标准 C 语言编写的程序都是以 main()作为开头，指定了 C 程序执行的起点，在 C 程序中只能出现一次，称为主函数
19.C 程序总是从 main 函数开始执行，与他在程序中的位置无关，main 函数主体部分称为语句，用{}括起来，一般 C 语句以;结束
20.变量的类型决定了编译器为其分配内存单元的字节数、数据在内存单元中的存放形式、该类型变量合法的取值范围以及该类型变量可参与的运算种类

中国计算机学会 www.51cs.com

21.变量名是用户定义的标识符，用于标识内存中具体的存储单元，存放的数据称为变量的值
22.新数据存放于存储单元时，旧数据将会被修改，反应变量的值是可以改变的
23.变量名的命名规则：
1)标识符只能由英文字母、数字、下划线组成
2)标识符必须以字母或下划线开头
3)不允许使用 C 关键字为标识符命名
4)标识符可以包含任意多个字符，但一般会有最大长度限制，预编译器有关，一般不会超过，最好不超过 8 个字符
注意：标识符区分大小写
eg:x y3 _imax ELSE X A_to_B (合法)
eg:7x int #No bad one re-input(不合法)
24.标准 C 规定所有变量必须在第一条可执行语句之前定义
25.在同一条语句中可以同时定义多个相同类型变量，多个变量之间用逗号分隔，没有顺序要求
26.在使用标准输入/输出函数时，必须在程序开头加上编译预处理命令
27.<stdio.h>中 stdio 为头文件，std 是 standard,i 为 input,o 为 output,h 为 head
28.编译预处理命令#include 的作用：将在输入/输出函数的头文件 stdio.h 包含到用户源文件中
29.%d 按十进制整数个格式输出，%f 按十进制小数格式输出，一般输出 6 位小数，%c 输出一个字符，\n 换行，双引号内字符原样输出
30.不同类型的数据在内存中占用不同大小的存储单元，他们所能表示的数据的取值范围各不相同，不同类型的数据表示形式及其可以参与的运算种类也不同
31.定义整型变量时，只要不指定为无符号型，其隐含类型为有符号型，signed 通常省略不写
32.C 程序中每个变量的值都是按字节编址，都被存储在内存中特定的存储单元中，这个存储空间实际是一个线性地址表，即每个字节的存储空间对应一个唯一的地址
33.1 个字节等于 8 个二进制位，即 8 个位为一个字节，一个字节可以表示的整数最小为 0，最大 255，即 8 个位可以表示 0-255 之间的数，一个二进制位的值只能是 0 或 1
34.同种类型在不同的平台所占字节数不同，要计算数据类型所占内存空间的字节数需要用 sizeof()运算符
35.sizeof 是 C 语言关键字，不是函数名，sizeof(变量名)计算一个变量所占内存的字节数
36.计算不同类型运算符的表达式时，要考虑运算符的优先级和结合性
37.正数的反码、补码与原码都相同，负数的补码等于反码加 1
38.在计算机内存中负数都是用补码来表示
39.对于实数，无论小数海曙指数形式，在计算机内存中都采用浮点形式来存储
40.浮点数分为阶码、尾数两部分，实数 N=S*pow(r,j),S 为尾数(无论正负，规定用存小数)，j 为阶码(无论正负，必须是整数)，r 是基数 eg:10.0111=0.100111*pow(2,10)
41.阶码所占位数决定实数的取值范围，尾数所占位数决定实数的精度，尾数的符号决定实数的正负，阶码和尾数的字节大小与编译器有关
42.float 型变量能接收实型常量的 7 位有效数字，double 型变量能接收 16 位
43.运算符的优先级：单目运算<算术运算<关系运算<逻辑运算<条件运算<赋值运算<逗号运算
44.素数：又称质数，指在大于 1 的自然数中，除 1 和本身不能被其他自然数整除的数

45.合数：指在自然数中除 1 和本身外还能被其他自然数整除的数
46.因子：所有能整除这个数的数，不包括自身，但包括 1
47.闰年：能被 4 整除但不能被 100 整除，或能被 400 整除

二、 简单的算术运算和表达式

1.条件运算符是 C 语言提供的唯一一个三元运算符， C 语言中没有幂运算符
2.只有计算相反数是一元运算符，其余运算符都是二元运算符
3.一元运算符也叫单目运算符，二元运算符也叫双目运算符，三元运算符也叫三目运算符
4.整数除法的结果是整数，浮点数除法的结果是浮点数
5.求余运算限定参与运算的两个操作数必须为整型，余数的符号与被除数的符号相同，不能对两个实型数进行求余运算
6.所有的算术运算符中只有一元的去相反数运算符为右结合，其余的结合性都为左结合
7.同一优先级的运算符进行混合运算时，从左向右依次进行
8.运算符的优先级：单目运算>算术运算>关系运算>逻辑运算>条件运算>赋值运算>逗号运算
9.计算并输出一个三位整数的个位、十位、百位数字之和

```
#include<stdio.h>
void main()
{
    int x=321;
    int a,b,c,sum;
    a=x%10;//a=1
    b=(x/10)%10;//b=2
    c=x/100;//c=3
    sum=a+b+c;
    printf("a=%d,b=%d,c=%d\nsum=%d\n",a,b,c,sum);
}
```

10.涉及算术运算的复合赋值运算符有 5 个：+=,-=,*=,/=,%=
11.增 1 运算符也称自增运算符，减 1 运算符也称自减运算符，都是一元运算符，只有一个操作数必须是变量不能是常量或表达式
12.自增自减运算符作为前缀时，先执行加 1 或减 1 然后使用；自增自减运算符作为后缀时，先使用再执行加 1 或减 1；
13.考虑优先级与结合性的实例
m=-n++《=》m=-(n++)《=》m=-n,n=n+1;
//正面：-和++都是一元运算符，优先级相同，此时要考虑结合性，结合性都是右结合的，所以先算++后算-
//反面：如果等价 m=(-n)++就不合法了，因为表达式不能进行自增操作
14.scanf 函数和 printf 函数都是 C 的标准输入/输出函数，&为取地址运算符
15.宏常量与宏替换：

1) 在程序中直接使用的常数称为幻数，将幻数定义为宏常量或 const 常量是为了提高程序的可读性和可维护性

2) 宏常量也称符号常量，没有数据类型，编译器不对其进行类型检查，只进行字符串替换

51514.COM

3) 宏定义一般形式：#define 标识符 字符串。标识符被称为宏名,宏名与字符串之间可有多个空白符，不加等号，结尾不加分号

4) 宏常量是由宏定义编译预处理命令来定义,宏名替换成字符串的过程称为宏替换，宏替换不做任何语法检查

5) 当宏定义是一个表达式时，调用时，是一个数就可以直接带入，而是表达式也要看成是一个数代进宏定义表达式中，

而看成一个数这就要求把调用的数加上圆括号，为了减少不必要的错误，最好都加上圆括号

6) 宏替换的过程是将宏名原样替换成字符串，而不是直接计算出值，所以要用调用后的结果参与其他运算就需要把调用的结果加上圆括号

7) 例题:

```
#include<stdio.h>
#define N 2                    //N=2
#define M N+1                  //M=2+1
#define NUM 2*M+1              //NUM=2*2+1+1=6
void main()
{
    int i;
    for(i=1;i<=NUM;i++);
    printf("%d\n",i); //执行完空循环，i=7
    i--;                       //  i=6
    printf("%d\n",i);
}
```

8) 例题:

```
#include<stdio.h>
#define f(x) x*x
void main()
{
    int b;
    b=f(4+4);                  //b=4+4*4+4=24，解释见 5)
    printf("b=%d\n",b);
}
```

9) 例题：

```
#include<stdio.h>
#define f(x) x*x
void main()
{
    int b;
    b=f(4+4)/(2+2);        //b=4+4*4+4/(2+2)=21 解释见 6)
    printf("b=%d\n",b);
}
```

10) 例题：

```
#include<stdio.h>
#define f(x) x*x
```

中考专升本网 www.515148.com

```
void main()
{
    int b;
    b=f((4+4));              //b=(4+4)*(4+4)=64
    printf("b=%d\n",b);
}
```

11) 例题：

```
#include<stdio.h>
#define f(x) (x*x)
void main()
{
    int b;
    b=f((4+4))/(2+2);     //b=((4+4)*(4+4))/(2+2)=16
    printf("b=%d\n",b);
}
```

16.const 常量

1) const 常量只能在定义时赋初值，因为编译器将其放在只读存储区，不允许在程序中修改

2) const 常量的定义一般形式：const 类型名 标识符=字符串；//将标识符声明为具有该数据类型的 const 常量

3) const 是一种类型修饰符，const 常量具有数据类型，编译器要对其进行类型检查

17.表达式中的自动类型转换：

1) 表达式中，操作数类型相同，运算的结果类型与操作数类型相同

2) 表达式中，操作数类型不相同，C 编译器自动将所有操作数进行类型提升，转换成同一类型，所占字节最大，再计算

18.赋值中的自动类型转换：

1) 赋值运算符左侧变量与右侧表达式类型不同时，发生自动类型转换：右侧表达式的值转换成左侧变量的类型

19.强制类型转换：

1) 强制类型转换运算符是一元运算符

2) 强制类型转换也称强转，是将表达式的值转换为任意类型，并不改变变量本身的数据类型

3) 强转一般形式：(类型)表达式

4) 演示强制类型转换运算符的使用

```
#include<stdio.h>
void main()
{
    int m=5;
    printf("m/2=%d\n",m/2);//m=2
    printf("(float)(m/2)=%f\n",(float)(m/2));//m=2.000000
    printf("(float)m/2=%f\n",(float)m/2);//m=2.500000
    printf("m=%d\n",m);//m=5,只是将 m 强转为实型数据，并不改变 m 的数据类型
}
```

20.常用的标准数学函数：

中国软件开发网 WWW.515148.COM

1) 使用 C 标准数学函数，要在程序开头加上编译预处理命令：#include<math.h>

2) 例：已知三角形三边长为 a,b,c，计算三角形的面积

```
#include<stdio.h>
#include<math.h>
void main()
{
    float a,b,c,s,area;
    printf("Input a,b,c:");
    scanf("%f,%f,%f",&a,&b,&c);
    s=(a+b+c)/2;
    area=(float)sqrt(s*(s-a)*(s-b)*(s-c));
    printf("area=%f\n",area);
}
```

21.赋值运算符的左侧不能出现表达式，变量与变量之间可以赋值

例 1：若有定义：int a,b,c;下列表达式中哪一个是合法的 C 语言赋值表达式（C、D）

A)a=7+b=c=7　　B)a=b++=c=7　　C)a=(b=7,c=12)　　D)a=3,b=a+5,c=b+2

//A.7+b 表达式不能做左值　B.b++表达式不能做左值　C.逗号表达式 a=12　D.依次赋值

例 2：下面不正确的赋值语句是(B)

A)a=++a　　B)a=b++=10　　C)a+=b　　D)a=1>'a'

//D.算术运算符优先级高于赋值运算符，先算 1>'a'再将其结果赋给 a

例 3：若有下列定义：int a=3,b=2,c=1;以下选项错误的赋值表达式是(A)

A)a=(b=4)=3　　B)a=b=c+1　　C)a=(b=4)+c　　D)a=1+(b=c=4)

//A.先计算圆括号里的值等于 4，然后按照赋值表达式的顺序从右向左计算，将 3 赋值给 4 这是不对的，

即赋值号左侧只能是变量，不能出现表达式 b=4

甘肃专升本网 WWW.516148.COM

三、 键盘输入和屏幕输出

1.字符常量：

把字符放在一对单引号内，适用于多数可打印字符

2.转义字符：

以反斜线(\)开头，也是放在一对单引号内，适用于控制字符(如回车符，换行符)

3.常用的转义字符：

1) '\n' — 换行
2) '\r' — 回车(不换行)
3) '\0' — 空字符
4) '\t' — 水平制表
5) '\v' — 垂直制表
6) '\b' — 退格
7) '\f' — 走纸换页
8) '\a' — 响铃报警提示音
9) '\"' — 一个双引号
10) '\'' — 一个单引号
11) '\\' — 一个反斜线
12) '\?' — 问号
13) '\ddd' —1 到 3 位八进制 ASCII 码值所代表的字符
14) '\xhh' —1 到 2 位十六进制 ASCII 码值所代表的字符

4.\n,是将光标移到下一行起始位置，\r 是将光标移到当前行的起始位置

5.\t,是水平制表符，相当于按下 Tab 键，每次按下 Tab 键并不是从当前光标位置向后移动一个 Tab 宽度，而是移到下一个制表位

实际移动的宽度视当前光标位置距相邻的下一个制表位的距离而定

6.制表位，屏幕上的一行被分为若干个域，相邻域之间的交接点称为制表位，每个域的宽度就是一个 Tab 宽度，多数习惯上为 4

7.当转义序列出现在字符串中时，是按单个字符计数的

8.一个整形数在内存中是以二进制形式存储的，而一个字符在内存中也是以其对应的 ASCII 码的二进制形式存储的，

但 char 型数据在内存中只占一个字节，而 int 型数据在 16 位系统中占 2 个字节，32 位系统占 4 个字节

9.在 ASCII 码取值范围内，char 型数据可以和 int 型数据进行混合运算，char 型数据也能以 int 型输出，直接输出其对应的 ASCII 码的十进制值

10.字符的输入/输出：

1) getchar()和 putchar()是 C 标准函数库中专门用于字符输入/输出的函数，功能是只能输入/输出一个字符

2) 例：从键盘输入一个大写英文字母，将其转换为小写字母显示在屏幕上

```
#include<stdio.h>
void main()
{
    char ch;
    ch=getchar();//键盘输入一个字符，再按回车表示输入结束，字符存入变量 ch，注意：getchar()没有参数，直接返回读入的字符
    ch=ch+32;
    putchar(ch);//第二次按回车，将显示转换后的结果
    putchar('\n');//注意：putchar()内一定要有参数，参数就是待输出的字符，可以是可打印字符，也可是转义字符
}
```

11.数据的格式化屏幕输出：

1) 函数 printf()的一般格式：printf(格式控制字符串);或者 printf(格式控制字符串,输出值

甘肃长大网 WWW.55148.COM

参数表);

2) 格式控制字符串包括：格式转换说明符，需要原样输出的普通字符

3) 函数 printf()的格式转换说明符：

%d — 输出带符号十进制整数，整数符号省略

%u — 输出无符号十进制整数

%o — 输出无符号八进制整数，不输出前导符 0

%x — 输出无符号十六进制整数（小写），不输出前导符 0x

%X — 输出无符号十六进制整数（大写），不输出前导符 0x

%c — 输出一个字符

%s — 输出字符串

%f — 以十进制小数形式输出实数，包含单，双精度，隐含 6 位小数，但并非全是有效数字，单精度有效 7 位，双精度 16 位

%e — 以指数形式输出实数（小写 e 表示指数部分），小数点前有且仅有一位非 0 数字

%E — 以指数形式输出实数（大写 E 表示指数部分）

%g — 自动选取 f 或 e 格式中宽度较小的一种使用，不输出无意义的 0

%% — 输出%

4) 输出值参数表：需要输出的数据项的列表，可以是变量或表达式，逗号隔开，类型与格式转换说明符相匹配

5) 每个格式转换说明符与输出值参数表中的输出值参数一一对应，没有输出值参数，格式控制字符串就不需要格式转换说明符

6) 例：从键盘输入一个大写英文字母，将其转换为小写字母和其十进制 ASCII 码值显示在屏幕上

```
#include<stdio.h>
void main()
{
    char ch;
    printf("Please enter a key:");
    ch=getchar();
    ch=ch+32;
    printf("%c,%d\n",ch,ch);//printf("%c",ch)<<=>>putchar(ch),printf("\n")<<=>>putchar('\n')
}
```

7) 函数 printf()中的格式修饰符：在%和格式符中间插入格式修饰符，用于输出格式的微调，如：指定输出域宽、精度、左对齐等

英文字母 l — 修饰格式符 d,o,x,u 时，用于输出 long 型数据

英文字母 L — 修饰格式符 f,e,g 时，用于输出 long double 型数据

英文字母 h — 修饰格式符 d,o,x 时，用于输出 short 型数据

输出域宽 m — m 为整数，输出时占 m 列，若 m>0，当数据宽度小于 m 时，域内右靠齐，左补空格，当数据宽度大于 m 时，修饰符失效，按实际宽度输出，若 m 有前导符 0，左边多余位补 0；若 m<0,输出数据在域内左靠齐

显示精度 .n — n 为大于等于 0 的整数，精度修饰符位于最小域宽修饰符之后，由圆点和整数构成，对于浮点数，用于指定输出的浮点数小数位数；对于字符串，用于指定从字符串左侧开始截取的子串字符个数

8) 使用 const 常量定义 pi,编程从键盘输入圆的周长和面积，使其输出数据保留两位小数

甘肃专升本网 www.515148.com

```
#include<stdio.h>
void main()
{
    const double pi=3.14159;
    double r,circum,area;
    printf("Please enter r:");
    scanf("%lf",&r);
    circum=2*pi*r;
    area=pi*r*r;
    printf("输出没有宽度和精度的值:");
    printf("circum=%f,area=%f\n",circum,area);
    printf("输出没有宽度和精度的值:");
    printf("circum=%7.2f,area=%7.2f\n",circum,area);//域宽为 7，占 7 个字符宽度，小数点也算一个字符，精度为 2，小数点后保留 2 位
}
```

12.数据的格式化键盘输入：

1) 函数 scanf()的一般格式：scanf(格式控制字符串，参数地址表);
2) 格式控制字符串：包括格式转换说明符，分隔符
3) 格式转换说明符以%开始，以格式字符结束，用于指定各参数的输入格式
4) 函数 scanf()的格式转换说明符：
   %d — 输入十进制整数
   %o — 输入八进制整数
   %x — 输入十六进制整数
   %c — 输入一个字符，空白字符(包括空格、回车、制表符)也作为有效字符输入
   %s — 输入字符串，遇到第一个空白字符(包括空格、回车、制表符)时结束
   %f 或%e — 输入实数，以小数或指数形式输入均可
   %% — 输入一个%
5) 参数地址表：由若干变量的地址组成的列表，用逗号分隔
6) 函数 scanf()中的格式修饰符：在%和格式符中间插入格式修饰符
   英文字母 l — 加在格式符 d,o,x,u 之前，用于输入 long 型数据；加在 f,e 之前，用于输入 double 型数据
   英文字母 L — 加在格式符 f,e 之前，用于输入 long double 型数据
   英文字母 h — 加在格式符 d,o,x 时，用于输入 short 型数据
   输出域宽 m — m 为正整数，指定输入数据的宽度，系统自动按此宽度截取所需数据
   显示精度 .n — n 为 0 或正整数，scanf()没有精度格式修饰符，输入时不能规定精度
   忽略输入修饰符* — 表示对应的输入项在读入后不赋给相应的变量
7) 函数 scanf()输入数值型数据时，被认为输入结束的几种情况：遇空格符、回车符、制表符；达到输出域宽；遇非法字符输入
8) 如果函数 scanf()的格式控制字符串中存在除格式转换说明符以外的其他字符，必须将这些字符原样输入

```
#include<stdio.h>
void main()
{
```

计算机技术网 www.515148.com

```
    int a,b;
    scanf("%d %d",&a,&b);        //以空格作为分隔符，等价于 scanf("%d%d",&a,&b),等价于 scanf("%d    %d",&a,&b)，
    printf("a=%d,b=%d\n",a,b);//实际输入时，空格、回车符、制表符都可作为分隔符，即输入:12 空格 34  12 回车 34  12 制表符 34 均可，
}

#include<stdio.h>
void main()
{
    int a,b;
    scanf("%d,%d",&a,&b);//输入时必须以逗号作为分隔符，即只能输入：12,34
    printf("a=%d,b=%d\n",a,b);
}

#include<stdio.h>
void main()
{
    int a,b;
    scanf("a=%d,b=%d",&a,&b);//输入时必须将"a="和"b="原样输入，即只能输入：a=12,b=34
    printf("a=%d,b=%d\n",a,b);
}

#include<stdio.h>
void main()
{
    int a,b;
    scanf("%2d%2d",&a,&b);//输入时将前两位数读入存到 a 中，接着两位存到 b 中，然后结束读取，即输入：1234，或 123456 结果一样
    printf("a=%d,b=%d\n",a,b);
}

#include<stdio.h>
void main()
{
    int a,b;
    scanf("%d %d",&a,&b);
    printf("a=\"%d\",b=\"%d\"\n",a,b);//输出 a="12",b="34"
}

#include<stdio.h>
void main()
{
    int a,b;
```

甘肃考试网 www.515148.com

```
    scanf("%d%*c%d",&a,&b);//任意字符都可作为分隔符,即输入：12,34 或 12？34 或 12+34 结果都一样
    printf("a=%d,b=%d\n",a,b);
}

#include<stdio.h>
void main()
{
    int a,b;
    scanf("%2d%*2d%d",&a,&b);//忽略两位，输入 123456
    printf("a=%d,b=%d\n",a,b);//输出 a=12,b=56
}
```

对于程序

```
#include<stdio.h>
void main()
{
    int a,b;
    scanf("%d %d",&a,&b);
    printf("a=%d,b=%d\n",a,b);
}
```

如果输入 12 34a 回车，则结果 a=12,b=3,程序在遇到非法字符 a 时，会导致程序输入终止，此时 a 会读入 12，b 会读入 3

如果输入 123a 回车，则结果 a=123,b=-858993460,程序在遇到非法字符 a 时，会导致程序输入终止，此时 a 会读入 123，而 b 未能读入指定数据项数

如果在 scanf()函数中忘记在变量前加取地址运算符&，会导致非法内存访问

13.%c 格式符使用几种情况：

1) 用%c 格式读入字符时，空格字符和转义字符（包括回车）都会被当做有效字符读入

例：键盘输入一个整数加法算式：操作数 1+操作数 2，输出：操作数 1+操作数 2=计算结果

```
#include<stdio.h>
void main()
{
    int data1,data2;
    char  op;                                   //输入：12  空格+空格  3      回车//输出：12-858993460=-858993448
    printf("Enter data1+data2\n");//输入：12 空格 3              回车//输出：12 3=15
    scanf("%d%c%d",&data1,&op,&data2);//输入：12+3          回车//输出：12+3=15
    printf("%d%c%d=%d\n",data1,op,data2,data1+data2);
}
```

2) 先输入一个数据后再输入字符型变量时，输入一个数据后，输入的回车符将被当做有效字符读给字符型变量

例：编程从键盘先后输入 int 型、char 型和 float 型数据，要求每输入一个数据就显示这个数据的类型和数据值

计算专业本科 WWW.515148.COM

```
#include<stdio.h>
void main()
{
    int a;
    char b;
    float c;
    printf("Please input an integer:");
    scanf("%d",&a);
    printf("integer:%d\n",a);
//在输入一个整型数据后，输入的回车符被当做有效字符读给了字符型变量 b 了
    printf("Please input a character:");
    scanf("%c",&b);
    printf("chracter:%c\n",b);

    printf("Please input a float number:");
    scanf("%f",&c);
    printf("float:%f\n",c);
}
```

14.%c 格式符存在问题的解决方法

1) 用函数 getchar()将数据输入时存入缓冲区中的回车符读入，以避免被后面的字符型变量作为有效字符读入

```
#include<stdio.h>
void main()
{
    int a;
    char b;
    float c;
    printf("Please input an integer:");
    scanf("%d",&a);
    printf("integer:%d\n",a);
    getchar();//将存于缓冲区中的回车符读入，避免在后面作为有效字符读入
    //函数 getchar()的返回值是一个回车符，已经避免了错误，不需要再将其赋给字符型变量使用

    printf("Please input a character:");
    scanf("%c",&b);
    printf("chracter:%c\n",b);

    printf("Please input a float number:");
    scanf("%f",&c);
    printf("float:%f\n",c);
}
```

2) 在%c 前面加一个空格，将前面数据输入时存于缓冲区的回车符读入，避免被后面的字符型变量作为有效字符读入

甘肃软件本网 www.515148.com

```
#include<stdio.h>
void main()
{
	int a;
	char b;
	float c;
	printf("Please input an integer:");
	scanf("%d",&a);
	printf("integer:%d\n",a);

	printf("Please input a character:");
	scanf(" %c",&b);//在%c 前面加空格，将存于缓冲区中的回车符读入
	printf("chracter:%c\n",b);

	printf("Please input a float number:");
	scanf("%f",&c);
	printf("float:%f\n",c);
}
```

//解释：在%c 前加空格为什么能将存于缓冲区中的回车符读入

因为如果函数 scanf()的格式控制字符串中存在除格式转换说明符以外的其他字符，必须将这些

字符原样输入，所以在%c 前加空格，就必须在输入字符型数据前先原样输入空格，而空格，回车符，制表符在

函数 scanf()输入数值型数据时都代表输入结束，由实践可知，空格符、回车符、制表符在输入时等效

所以，缓冲区中的回车将代替需要原样输入的空格，因此，实际上，在%c 前增加空格或者 Tab 键都可以完成，

并且与增加的数量无关，且可以混合增加

3）*经典例题：编程实现键盘任意输入字符，统计字母、数字、空格、其他字符数量

首选字符串的方法：

```
#include<stdio.h>
#define N 100
int main()
{
	char a[N];
	int i,m=0,n=0,b=0,c=0;
	printf("请输入字符：\n");
	gets(a);
	for(i=0;a[i]!='\0';i++)
	{
		if((a[i]>='a'&&a[i]<='z')||(a[i]>='A'&&a[i]<='Z')) m++;
		else if(a[i]>=0&&a[i]<=9) n++;
		else if(a[i]==' ') b++;
		else c++;
```

甘肃专业本网 WWW.115148.COM

```
    }
    printf("字母个数：%d\n",m);
    printf("数字个数：%d\n",n);
    printf("空格个数：%d\n",b);
    printf("其他字符个数：%d\n",c);
    return 0;
}
```

单个字符输入的方法，但是要解决缓冲区的问题

```
#include<stdio.h>
int main()
{
    char ch;
    int a,b,c,d;
    a=b=c=d=0;
    printf("请输入字符:\n");
    do{
        ch=getchar();
        getchar();//清除缓冲区的回车键，不然会当做第二个字符读入
        if(ch!='    ')//用 Tab 键控制输入结束，他在程序中只会出现一次并且统计一次，
        {           //然后结束，所以要去掉它，可以使用 if 语句，也可在前面初始化为 d=-1
            if((ch>='a'&&ch<='z')||(ch>='A'&&ch<='Z')) a++;
            else if(ch>='0'&&ch<='9') b++;
            else if(ch==' ') c++;
            else d++;
        }
    }while(ch!='    ');
    printf("字母个数：%d\n",a);
    printf("数字个数：%d\n",b);
    printf("空格个数：%d\n",c);
    printf("其他字符个数：%d\n",d);
    return 0;
}
```

//解释：你可以把 Tab 控制结束改成其他任意字符，包括字母、数字、/;都可以，
就是不能直接使用回车控制结束，因为你在实际键盘输入时，需要打回车才能把前面的字符读入，当你输完回车后，就已经换行了，再回车就达不到你要的效果了，不可能把他读入，但是他会留在缓冲区下一次使用，所以说，理论上任意字符都可以控制结束，但是不能直接使用回车再回车的方法，而 getchar()函数一次只能读一个字符，当你回车读入字符后，回车符就会留在缓冲区下一次使用，你可以试试，你把 getchar()这行语句注释，然后一次输入两个字符再回车的话，那么这时他也只能读入第一个字符，第二个字符和回车就会计入第二次、第三次的结果。

总结：这种方式与用字符串的区别在于，字符串可以统计任何字符，但是这种方式不能统计你用来控制结束的字符，比如说，你用/控制结束，那么就不能统计/的数量了，而且你要把他去掉，因为在整个程序中/只会出现一次，然后就结束了

计算机等级考试网 WWW.JSJ5148.COM

**当程序中出现两次或以上的键盘输入时，就有可能出现缓冲区的问题，只有一次输入，回车直接结束没有这种问题

15.考点题型：字符串常量的长度问题：

1)字符串常量"ab\\c\td\376"的长度是（A）

A) 7　　B) 12　　C) 8　　D) 14

// a　b　\\　c　\t　d　\376

2)语句 printf("%d",strlen("abs\no12\1\\"));的输出结果是（C）

A) 11　　B) 10　　C) 9　　D) 8

// a　b　s　\n　o　1　2　\1　\\

16.考点题型：赋值运算、关系运算的混合运算问题：

1)设 m,n,a,b,c,d 的值均为 0，执行(m=a==b)||(n=c==d),m,n 的值是（C）

A) 0,0　　B) 0,1　　C) 1,0　　D) 1,1

// "=="是算术运算符，"="是赋值运算符，优先级：算术运算符>赋值运算符，先判断 a==b?是否正确

正确则为真，把 1 赋给 m,"||"是或运算，前面已经为真，所以||后面的就不会再运算了，结果就为真

所以 m=1,n=0 不变

2)int a,b,m,n;初值依次为 4,3,2,1,经计算 m=(a<=b)&&(n=m<n)后，m,n 的值为（C）

A) 1,1　　B) 1,0　　C) 0,1　　D) 0,0

// a<=b 为假，&&是并，前面已经为假，后面就不会再运算了，结果就为假，把 0 赋给 m,即 m=0,n=1 不变

甘肃专升本网 www.515148.COM

四、 选择控制结构

1.沃思提出“数据结构+算法=程序”只在面向过程的语言（如 C 语言）成立
2.算法的正确性衡量方法：
1)有穷性 2)确定性 3)有效性 4)允许没有输入或者有多个输入 5)必须有一个或者多个输出
3.算法的描述方法：
1)自然语言描述 2)流程图描述 3)NS 结构化流程图描述 4)伪码描述
4.关系表达式:用作判断条件，结果只有真和假，(n%2!=0)等价于(n%2),0 表示假，非 0 表示真
5.分支结构：当条件 P 成立时，执行 A 操作，否则执行 B 操作；如果 B 操作为空，即为单分支结构；
如果 B 操作不为空，即为双分支结构；如果 B 操作中又包含另一选择结构，则构成多分支选择结构；
6.单分支例题：计算输出两整数最大值
单分支(一般形式)：if(表达式 P) 语句 A

```
#include<stdio.h>
void main()
{
    int a,b,max;
    printf("Enter a,b:");
    scanf("%d%d",&a,&b);
    if(a>b)                 //单分支：if(表达式 P) 语句 A
        max=a;
    if(a<=b)                //单分支：if(表达式 P) 语句 A
        max=b;
    printf("max=%d\n",max);
}
```

7.双分支例题：计算输出两整数最大值
双分支(一般形式):if(表达式 P) 语句 1
else 语句 2

```
#include<stdio.h>
void main()
{
    int a,b,max;
    printf("Enter a,b:");
    scanf("%d%d",&a,&b);
    if(a>b)
        max=a;
    else
        max=b;
    printf("max=%d\n",max);
}
```

8.条件运算符是 C 语言中唯一的三元运算符，需要三个操作数

甘肃考试本网 www.515148.com

9.条件运算符例题：计算输出两整数最大值
条件运算符(一般形式)：表达式 1？表达式 2：表达式 3
含义：若表达式 1 为非 0，则结果为表达式 2 的值，否则是表达式 3 的值

```
#include<stdio.h>
void main()
{
    int a,b,max;
    printf("Enter a,b:");
    scanf("%d%d",&a,&b);
    max=a>b?a:b;
    printf("max=%d\n",max);
}
```

10.多分支控制条件语句(一般形式)：

```
    if(表达式 1)  语句 1
    else if(表达式 2)  语句 2
    ...
    else if(表达式 m)  语句 m
    else    语句 m+1
```

11.条件语句在语法上只允许每个条件分支中带一条语句，而要在分支里处理多条语句就要使用花括号，构成复合语句
12.随机数的使用方法：
1) 符号常量 RAND_MAX 在头文件#include<stdlib.h>中定义，标准 C 规定 RAND_MAX 不大于双字节整数的最大值 32767
2）随机函数 rand()产生的是一个在 0~RAND_MAX 之间的整数,即[0,32767]之间的整数
3) 利用求余运算 rand()%b 可将函数 rand()生成的随机数变化到[0,b-1]
4) 利用 rand()%b+a 可将随机数的取值范围平移到[a,a+b-1]
例：产生 1~100 的随机数： rand()%100+1
13.例题：猜数游戏：计算机想一个数，人猜对了提示 Right!,否则提示 Wrong!，并告诉大小

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int magic,guess;
    magic=rand();                          //随机数
    printf("Please guess a number:");
    scanf("%d",&guess);
    if(guess>magic)                        //多分支
    {
        printf("Wrong!Too big!\n");   //复合语句
    }
    else if(guess<magic)
    {
        printf("Wrong!Too small!\n");
    }
```

北京专升本网 WWW.515148.COM

```
    else{
        printf("Right!\n");
        printf("The number is:%d\n",magic);
    }
}
```

14.switch 语句：用于多路选择，一般情况大于三种，用开关语句代替条件语句

1) switch 语句(一般形式)：

```
switch(表达式)
{
    case 常量 1：
            可执行语句序列 1     //注意：语句序列不使用花括号
    case 常量 2:
            可执行语句序列 2
    .....
    case 常量 n:
            可执行语句序列 n
    default:
            可执行语句序列 n+1
}
```

2) switch 语句相当于一系列的 if-else 语句

3）switch 后圆括号内的表达式只能是 char 型或 int 型

4）关键字 case 后接常量，类型与表达式类型一致，常量的值必须互不相同

5）常量与 case 中间至少有一个空格，常量后面是冒号

6）case 本身并没有条件判断的功能，若 case 后面的语句省略不写，则表示它与后续 case 执行相同的语句

7）switch 语句的执行过程：

第一步，计算 switch 后表达式的值

第二步，将表达式的值依次与 case 后的常量比较

第三步，如果相等，执行 case 后的代码段，执行完毕，可使用 break 语句跳出 switch 语句

如果没有 break 语句，不会再比较，程序将依次执行下面的 case 后的语句，直到遇到 break 或者 switch 的}为止

15.例题：编程实现简单计算器功能

```
#include<stdio.h>
void main()
{
    int data1,data2;
    char op;
    printf("Enter your expression:");
    scanf("%d%c%d",&data1,&op,&data2);
    switch(op)
    {
    case '+':
        printf("%d+%d=%d\n",data1,data2,data1+data2);
```

515148.COM

```
        break;
    case '-':
        printf("%d-%d=%d\n",data1,data2,data1-data2);
        break;
    case '*':
    case 'x':
    case 'X':
        printf("%d*%d=%d\n",data1,data2,data1*data2);
        break;
    case '/':
        if(0==data2)//也可写成 data2==0,写 0==data2 的好处：
        { //因为赋值表达式左值必须是变量，如果把==误写成=，会提示错误
            printf("Division by zero!\n");
        }
        else{
            printf("%d/%d=%d\n",data1,data2,data1/data2);
        }
        break;
    default:
        printf("Invalid operator!\n");
      //break;这句一般不需要，因为没有 break 就会依次执行下面的步骤直到 break 或者 switch 的}
    } //这里已经是 switch 的}了
}
```

16.例题：编程实现简单计算器功能，能进行浮点数运算，操作数与运算符之间可加入任意多个空格

```
#include<stdio.h>
#include<math.h>
void main()
{
    float data1,data2;
    char op;
    printf("Enter your expression:");
    scanf("%f %c%f",&data1,&op,&data2);
    switch(op)
    {
    case '+':
        printf("%f+%f=%f\n",data1,data2,data1+data2);
        break;
    case '-':
        printf("%f-%f=%f\n",data1,data2,data1-data2);
        break;
    case '*':
    case 'x':
```

北京专升本网 WWW.515148.COM

```
        case 'X':
            printf("%f*%f=%f\n",data1,data2,data1*data2);
            break;
        case '/':
            if(fabs(data2)<=1e-7)
            {
                printf("Division by zero!\n");
            }
            else{
                printf("%f/%f=%f\n",data1,data2,data1/data2);
            }
            break;
        default:
            printf("Invalid operator!\n");
        }
}
```

//内存中的浮点数所表示的精度是有限的，所以比较实型变量 data2 与 0 是否相等时，不能像整型变量与 0 比较那样直接用相等关系运算符与 0 比较，而应使用如下方式判断实型变量 data2 的值是否位于 0 附近的一个很小的区间内，即：if(fabs(data2)<=1e-7)

17.逻辑运算符和逻辑表达式

逻辑非：!,单目运算符，右结合, 优先级最高

逻辑与：&&,双目运算符，左结合，优先级较高

逻辑或：||,双目运算符，左结合，优先级较低

例：判断某一年 year 是否是闰年的条件：

1) 能被 4 整除，但不能被 100 整除；

1）能被 400 整除

逻辑表达式：((year%4==0)&&(year%100!=0))||(year%400==0)

或：(year%4==0)&&(year%100!=0)||(year%400==0)

18.特别注意：在计算含有逻辑运算符(&&和||)的表达式时，通常尽量使用最少的操作数来确定表达式的值，这就意味着表达式中的某些操作数可能不会被计算。

例：if(a>1&&b++>2&&c--!=0)中，仅在条件表达式 a>1 为真时，才会计算后面的值

19.程序测试：

1）程序测试只能证明程序有错，不能证明程序无错

2）例：编程输入三角形三边长 a,b,c,判断他们能否构成三角形，能构成什么三角形

```
#include<stdio.h>
#include<math.h>
#define EPS 1e-1
void main()
{
    float a,b,c;
    int flag=1;
    printf("Enter a,b,c:");
    scanf("%f,%f,%f",&a,&b,&c);
    if(a+b>c&&b+c>a&&a+c>b)
```

WWW.515148.COM

```
    {
        if(fabs(a-b)<=EPS&&fabs(b-c)<=EPS&&fabs(c-a)<=EPS)
        {
            printf("等边");
            flag=0;
        }
        else if(fabs(a-b)<=EPS||fabs(b-c)<=EPS||fabs(c-a)<=EPS)
        {
            printf("等腰");
            flag=0;
        }
        if(fabs(a*a+b*b-c*c)<=EPS||fabs(b*b+c*c-a*a)<=EPS||fabs(a*a+c*c-b*b)<=EPS)
        {
            printf("直角");
            flag=0;
        }
        if(flag)
        {
            printf("一般");
        }
        printf("三角形\n");
    }
    else{
        printf("不是三角形\n");
    }
}
```

3)例：编程将输入的百分制成绩转换为五分制成绩输出

```
#include<stdio.h>
void main()
{
    int score,mark;
    printf("Enter score:");
    scanf("%d",&score);
    if(score<0||score>100)
    {
        mark=-1;//这个 if 语句很重要，如果去掉，边界测试 101~109，-1~-9 会出错
    }
    else{
        mark=score/10;
    }
    switch(mark)
    {
    case 10:
    case 9:
```

甘肃专升本网 WWW.515148.COM

```
            printf("%d-优!\n",score);
            break;
        case 8:
            printf("%d-良!\n",score);
            break;
        case 7:
            printf("%d-中!\n",score);
            break;
        case 6:
            printf("%d-及格!\n",score);
        case 5:
        case 4:
        case 3:
        case 2:
        case 1:
        case 0:
            printf("%d-不及格!\n",score);
            break;
        default:
            printf("Input error!\n");
        }
}
```

20.对输入非法字符的检查与处理

1)例：输入两个整型数，计算并输出两个整数的最大值

```
#include<stdio.h>
void main()
{
    int a,b,max;
    printf("Enter a,b:");
    scanf("%d,%d",&a,&b);
    max=a>b?a:b;
    printf("max=%d\n",max);
}
//隐含问题：在 Visual C++下
1.如果输入 3.2,1 则输出 3
2.如果输入 1,3.2 则输出 3
3.如果输入 q 则输出-858993460
//解决办法：
#include<stdio.h>
void main()
{
    int a,b,max,ret;
    do{
        printf("Enter a,b:");
```

计算机等级考试本网 WWW.51514B.COM

```
        ret=scanf("%d,%d",&a,&b);
        if(ret!=2)//返回在遇到非法字符前已成功读入的数据项数
        {
        //但是 scanf()函数不做类型检查，所以输入 1,3.2 会输出 3
            printf("Input error!\n");
            while(getchar()!='\n');
        }
        else{
            max=a>b?a:b;
            printf("max=%d\n",max);
        }
    }while(ret!=2);
}
```

21.位运算符的使用

1）位运算是对字节或字内的二进制数位进行测试、抽取、设置或移位等操作

2) 位运算的操作对象只能是 char 和 int 类型

3) C 语言中共有 6 种位运算符，只有按位取反是单目运算符，其他都是双目运算符

4）关系运算和逻辑运算的结果要么是 0，要么是 1，而位运算结果可为任意值，但每一位都只能是 0 或 1

5）6 种位运算符：

按位取反	~	用于加密处理，一次求反加密，二次求反解密
左移位	<<	
右移位	>>	
按位与	&	用于对字节中某位清零
按位异或	^	
按位或	|	用于对字节中某位置 1

6）按位取反 ~1=0，~0=1

例：~5=-6

```
~  00000101
————————————
   11111010   是-6 的补码
```

7）按位与 0&0=0,0&1=0,1&1=1

例：将 15 的最低位不变，其余位清零 15&1=1

```
   00001111
&  00000001
————————————
   00000001
```

8）按位或 0|0=0,0|1=1,1|1=1

例：将 15 的最高位不变，其余位均置 1 15|127=127

```
   00001111
|  01111111
————————————
   01111111  是 127 的补码
```

9）按位异或 0^0=0,0^1=1,1^1=0

比特上升本网 WWW.51CTO.COM

例：3^5=6

00000011

^ 00000101

————————

00000110 是 6 的补码

10）左移位

x<<n 表示把 x 的每一位向左平移 n 位，右边空位补 0

例：将 15 左移 1、2、3 位的二进制补码如下

初始字节内容 00001111 对应十进制 15

左移 1 位字节内容 00011110 对应十进制 30

左移 2 位字节内容 00111100 对应十进制 60

左移 3 位字节内容 01111000 对应十进制 120

11）右移位

x>>n 表示把 x 的每一位向右移 n 位，当 x 为有符号数时，左边空位补符号位值—算术移位

当 x 为无符号位时，左边空位补 0—逻辑移位

例 1：将 15 右移 1、2、3 位的二进制补码如下

初始字节内容 00001111 对应十进制 15

右移 1 位字节内容 00000111 对应十进制 7

右左移 2 位字节内容 00000011 对应十进制 3

右移 3 位字节内容 00000001 对应十进制 1

例 2：将-15 右移 1、2、3 位的二进制补码如下

初始字节内容 11110001 对应十进制-15

右移 1 位字节内容 11111000 对应十进制-8

右左移 2 位字节内容 11111100 对应十进制-4

右移 3 位字节内容 11111110 对应十进制-2

12) 无论左移位还是右移位，从一端移走的位不移入另一端，移走的位的信息就丢失了

13）左移位和右移位可以分表代替整数的乘法和除法，左移 n 位相当于乘以 2^n,右移 n 为相当于除以 2^n

14) 例：写出运行结果

```
#include<stdio.h>
void main()
{
   int x=12,y=8;
   printf("%5d%5d%5d\n",!x,x||y,x&&y);
   printf("%5u%5d%5d\n",~x,x|y,x&y);
   printf("%5d%5d%5d\n",~x,x|y,x&y);
}
```

x 0000 0000 0000 1100

~x 1111 1111 1111 0011

~x 反码 1000 0000 0000 1100

~x 反码加一 1000 0000 0000 1101 -13

输出： 0 1 1

4294967283 12 8

-13 12 8

北京专升本网 www.515148.COM

五、循环控制结构

1.循环：包括计数控制的循环和条件控制的循环

2.结构化程序设计的三种基本结构：顺序结构、选择结构、循环结构

3.循环结构的类型：

1）当型循环结构：for 语句(适合循环次数已知,计数控制的循环)

2）直到型循环结构：while 语句、do-while 语句(适合循环次数未知,条件控制的循环)

4.while 语句

1）一般形式为：

```
while(循环控制表达式)
{                   |
    语句系列        |循环体
}                   |
```

2）循环控制表达式在执行循环体之前测试

3）执行过程：

计算循环控制表达式的值

如果循环控制表达式的值为真，那么执行循环体中的语句，并返回步骤 1

如果循环控制表达式的值为假，就退出循环，执行循环体后面的语句

5.do-while 语句

1）一般形式为：

```
do
{                   |
    语句系列        |循环体
}while(循环控制表达式);
```

2）循环控制表达式在执行循环体之后测试

3）执行过程：

执行循环体中的语句

计算循环控制表达式的值

如果循环控制表达式的值为真，那么返回步骤 1

如果循环控制表达式的值为假，就退出循环，执行循环体后面的语句

6.for 语句

1）一般形式为：

```
for(初始化表达式;循环控制表达式;增值表达式)
{                   |
    语句系列        |循环体
}                   |
```

2) 在每次循环体被执行之前，都要对循环控制条件测试一次，每次循环体执行完以后都要执行一次增值表达式

3）初始化表达式作用：循环变量初始化，即赋初值

4）循环控制表达式：是控制继续执行的条件，当表达式的值为非 0 时重复执行循环

5）增值表达式作用：每执行一次循环，循环控制变量增值

6）for 语句三个表达式之间用分号分隔，有且只能有两个分号

7）循环控制表达式若省略，表示循环条件为永真

8）初始化表达式和增值表达式都可省略，但是必须有其他语句反应其作用

甘肃老区技术网 WWW.516148.COM

7.例：从键盘输入 n,然后计算输出 1+2+3+……n 的值

1）while 语句编程实现

```
#include<stdio.h>
void main()
{
    int i=1,n,sum=0;//sum 一定要初始化，不然会是随机值
    printf("Enter n:");
    scanf("%d",&n);
    while(i<=n)
    {
        sum+=i;
        i++;
    }
    printf("1+2+3+……+%d=%d\n",n,sum);
}
```

2）do-while 语句编程实现

```
#include<stdio.h>
void main()
{
    int i=0,n,sum=0;
    printf("Enter n:");
    scanf("%d",&n);
    do
    {
        sum+=i;
        i++;
    }while(i<=n);
    printf("1+2+3+……+%d=%d\n",n,sum);
}
```

3）for 语句编程实现

```
#include<stdio.h>
void main()
{
    int i=1,n,sum=0;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        sum+=i;
    }
    printf("1+2+3+……+%d=%d\n",n,sum);
}
```

8.逗号运算符

1）所有运算符中优先级最低，左结合

北京电子开本网 WWW.515148.COM

2）作用：可实现对各个表达式的顺序求值

3）结果：将最后一个表达式的值作为整个逗号表达式的值

4）例：从键盘输入 n,然后计算输出 1+2+3+……n 的值

```
#include<stdio.h>
void main()
{
    int i,j,n,sum=0;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1,j=n;i<=j;i++,j--)
    {
        sum+=i+j;
    }
    printf("1+2+3+……+%d=%d\n",n,sum);
}
```

9.空语句

1）仅由一个分号构成的语句

2）作用：什么也不做，只起延时作用

3）例：

```
for(i=1;i<50000000;i++)
    {
        ;
    }
```

或

```
for(i=1;i<50000000;i++)
    {

    }
```

或

```
for(i=1;i<50000000;i++);

for(i=1;i<=n;i++);
    {
        sum+=i;
    }
```

等价于

```
for(i=1;i<=n;i++)
    {
        ;
    }
    sum+=i;
```

4) 例：从键盘输入 n,然后计算输出 1+2+3+……n 的值

```
#include<stdio.h>
void main()
{
    int i,j,n,sum=0;
    printf("Enter n:");
```

甘肃专升本网 WWW.515148.COM

```
    scanf("%d",&n);
    for(i=1,j=n;i<=j;i++,j--);
    {
        sum+=i+j;
    }
    printf("1+2+3+……+%d=%d\n",n,sum);
}//输出结果：101
```

10.死循环

1）while 语句行末加分号将导致死循环

2）例:i=1;

```
        while(i<=n);//行末加分号导致死循环
        {
            sum+=i;
            i++;
        }
```

相当于

```
        i=1;
        while(i<=n)//当输入大于 1 的 n 值，循环体中没有语句可以改变控制变量 i
        {               //使得循环条件为永真，导致死循环
            ;
        }
        sum+=i;
        i++;
```

11.while 语句和 do-while 语句的区别

1）while 先判断后执行，有可能一次都不执行，do-while 先执行后判断，至少执行一次

2）例：

```
n=101;                          n=101;
while(n<100)                    do
{                               {
    printf("n=%d\n",n);             printf("n=%d\n",n);
    n++;                            n++;
}                                   }while(n<100);
//循环一次也不执行              //结果：输出 n=101，循环执行一次
```

3）例：分别用 while 和 do-while 语句编程，输入一组数据，然后显示每次输入数据进行累加运算的结果，输入 0 结束

do-while 语句实现：

```
#include<stdio.h>
void main()
{
    int num,sum=0;
    do{
        printf("Enter num:");
        scanf("%d",&num);
        sum+=num;
```

比特考试网 www.315148.com

```
            printf("sum=%d\n",sum);
        }while(num!=0);
}
```

while 语句实现

```
#include<stdio.h>
void main()
{
    int num=1,sum=0;//给 num 赋任意非 0 值都可以
    while(num!=0)
    {
        printf("Enter num:");
        scanf("%d",&num);
        sum+=num;
        printf("sum=%d\n",sum);
    }
}
```

12.计数控制的循环：循环次数事先已知的循环

1）例 1：编写一个程序，从键盘输入 n,计算输出 n!

```
#include<stdio.h>
void main()
{
    int i,n,sum=1;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        sum*=i;
    }
    printf("%d!=%ld\n",n,sum);
}
```

2）例 2：编写一个程序，从键盘输入 n,计算输出 1!,2!……n!

```
#include<stdio.h>
void main()
{
    int i,n,sum=1;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        sum*=i;
        printf("%2d!=%ld\n",i,sum);
    }
}
```

3）例：键盘输入 n,编程计算 1!+2!+3!+……+n!

甘肃专升本网 WWW.515148.COM

```
#include<stdio.h>
void main()
{
    int term=1,sum=0,i,n;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        term*=i;
        sum+=term;
    }
    printf("1!+2!+3!+……+%d!=%ld\n",n,sum);
}
```

13.条件控制的循环：循环次数事先未知，由条件控制

1）例 1：输入两个整型数，计算并输出两个整型数的最大值，如若输入非法字符，提示错误并重新输入

//解释：非法字符，需要输入数字时，字符相对它来说就是非法字符，而需要输入字符型，数字对它不是非法字符，所有都是合法

```
#include<stdio.h>
void main()
{
    int a,b,max,ret;
    printf("Enter a,b:");
    ret=scanf("%d %d",&a,&b);
    while(ret!=2)//判断数据个数或格式是否错误
    {
        while(getchar()!='\n');//清除缓冲区中的错误数据
        printf("Enter a,b:");
        ret=scanf("%d %d",&a,&b);
    }
    max=a>b?a:b;
    printf("max=%d\n",max);
}
```

//注意：scanf()函数不做参数类型匹配检查，当输入 1 3.2 时，scanf 返回 2，不会导致重新输入

但是，此后的小数点仍留在缓冲区，如果此后还需输入内容，就要先清除缓冲区内容

2）例 2：先由计算机想一个 1-100 之间的数请人猜，若猜对，则计算机提示 Right!结束游戏，否则提示 Wrong!,并告诉人是大是小，直到猜对为止，记录人猜的次数，反应猜数的水平

```
#include<stdio.h>
#include<stdlib.h>
void main()
{
    int magic;
```

计课专用 WWW.515148.COM

```
    int guess;
    int counter=0;
    magic=rand()%100+1;
    do{
        printf("Please guess a magic number:");
        scanf("%d",&guess);
        counter++;
        if(guess>magic)
        {
            printf("Wrong!Too big!\n");
        }
        else if(guess<magic)
        {
            printf("Wrong!Too small!\n");
        }
        else{
            printf("Right!\n");
        }
    }while(guess!=magic);
    printf("counter=%d\n",counter);
}
```

14.随机数的使用

a.随机数的产生： rand()

1) 符号常量 RAND_MAX 在头文件#include<stdlib.h>中定义，标准 C 规定 RAND_MAX 不大于双字节整数的最大值 32767

2）随机函数 rand()产生的是一个在 0~RAND_MAX 之间的整数,即[0,32767]之间的整数

3) 利用求余运算 rand()%b 可将函数 rand()生成的随机数变化到[0,b-1]

4) 利用 rand()%b+a 可将随机数的取值范围平移到[a,a+b-1]

b.随机数的产生： srand(time(NULL))

1）用 rand()直接产生的随机数只是一个伪随机数，反复调用产生的随机数序列是一样的，而且每次都只用第一个

2）随机化：使程序每次运行时产生不同的随机数序列的过程

3）随机化的实现：通过调用标准库函数 srand()为函数 rand()设置随机数种子来实现

4）随机数种子的设置：

法 1：每次通过用户输入来完成随机化，srand(1),srand(2),srand(seed),输入 seed

法 2：的通过函数 time()读取计算机的时钟值，并把该值设置为随机数种子 srand(time(NULL))

5）函数 time()返回以秒计算的当前时间值，即一个代表时间的字符串，使用 NULL 作为 time()的参数时，

time(NULL)的返回值被转换成一个无符号整数，可作为随机数发生器的种子

6）使用 time()函数时，必须在程序开头将头文件<time.h>包含到程序中

北京老男孩 WWW.515148.COM

15.例：先由计算机想一个 1-100 之间的数请人猜，若猜对，则计算机提示 Right!结束游戏，否则提示 Wrong!,并告诉人是大是小，直到猜对为止，记录人猜的次数，反应猜数的水平

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void main()
{
    int magic;
    int guess;
    int counter=0;
    srand(time(NULL));
    magic=rand()%100+1;
    do{
        printf("Please guess a magic number:");
        scanf("%d",&guess);
        counter++;
        if(guess>magic)
        {
            printf("Wrong!Too big!\n");
        }
        else if(guess<magic)
        {
            printf("Wrong!Too small!\n");
        }
        else{
            printf("Right!\n");
        }
    }while(guess!=magic);
    printf("counter=%d\n",counter);
}
```

16.例：先由计算机想一个 1-100 之间的数请人猜，若猜对，则计算机提示 Right!屏幕输出多少次成功，结束游戏，否则提示 Wrong!,并告诉人是大是小，最多猜 10 次，超过就结束,要避免非法字符的输入

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void main()
{
    int magic;
    int guess;
    int counter=0;
```

www.515148.com

```
    int ret;//保存 scanf()函数的返回值
    srand(time(NULL));
    magic=rand()%100+1;
    do{
        printf("Please guess a magic number:");
        ret=scanf("%d",&guess);
        while(ret!=1)//若存在输入错误，则重新输入
        {
            while(getchar()!='\n');//清楚缓冲区中的内容
            printf("Please guess a magic number:");
            ret=scanf("%d",&guess);
        }               //若存在非法字符，则重新输入
        counter++;
        if(guess>magic)
        {
            printf("Wrong!Too big!\n");
        }
        else if(guess<magic)
        {
            printf("Wrong!Too small!\n");
        }
        else{
            printf("Right!\n");
        }
    }while(guess!=magic&&counter<10);
    printf("counter=%d\n",counter);
}
```

延伸拓展：先由计算机想一个 1-100 之间的数请人猜，若猜对，则计算机提示 Right!屏幕输出多少次成功，
结束游戏，否则提示 Wrong!,并告诉人是大是小，最多猜 10 次，超过就继续猜下一个数,每次运行程序，可
反复猜多个数，直到操作者想停时结束，要注意避免非法字符输入的问题

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void main()
{
    int magic;
    int guess;
    int counter;
    char reply;//保存用户输入的答案
    int ret;//保存 scanf()函数的返回值
    srand(time(NULL));
```

计算机等级考试本网 WWW.51514B.COM

```
    do{
        magic=rand()%100+1;
        counter=0;
        do{
            printf("Please guess a magic number:");
            ret=scanf("%d",&guess);
            while(ret!=1)//若存在输入错误，则重新输入
            {
                while(getchar()!='\n');//清楚缓冲区中的内容
                printf("Please guess a magic number:");
                ret=scanf("%d",&guess);
            }       //若存在非法字符，则重新输入
            counter++;
            if(guess>magic)
            {
                printf("Wrong!Too big!\n");
            }
            else if(guess<magic)
            {
                printf("Wrong!Too small!\n");
            }
            else{
                printf("Right!\n");
            }
        }while(guess!=magic&&counter<10);
        printf("counter=%d\n",counter);
        printf("Do you want to continue(Y/N or y/n)?");
        scanf(" %c",&reply);//%c 前有一个空格，读取缓冲区中的回车符
    }while(reply=='Y'||reply=='y');
}
```

17.设计一个简单计算器，允许连续做多次算术运算

```
#include<stdio.h>
#include<math.h>
void main()
{
    float data1,data2;
    char op;
    char reply;
    do{
        printf("Please enter your expression:\n");
        scanf("%f  %c%f",&data1,&op,&data2);//加空格可在操作数和运算符之间加任意多个空白符
        switch(op)
```

甘肃考试网 www.515148.com

```
        {
        case '+':
            printf("%f+%f=%f\n",data1,data2,data1+data2);
            break;
        case '-':
            printf("%f-%f=%f\n",data1,data2,data1-data2);
            break;
        case '*':
        case 'X':
        case 'x':
            printf("%f*%f=%f\n",data1,data2,data1*data2);
            break;
        case '/':
            if(fabs(data2)<=1e-7)
            {
                printf("Division by zero!\n");
            }
            else{
                printf("%f/%f=%f\n",data1,data2,data1/data2);
            }
            break;
        default:
            printf("Invalid operator!\n");
        }
        printf("Do you want to continue(Y/y or N/n)?");
        scanf(" %c",&reply);//加空格清除缓冲区中的回车符
    }
    while(reply=='Y'||reply=='y');
}
```

18.嵌套循环

一个循环语句放在另一个循环语句中构成的循环称为嵌套循环

1) 嵌套循环的总循环次数等于外层循环次数和内层循环次数的乘积

2）为避免造成混乱，嵌套循环的内层和外层的循环控制变量不要同名

3）例：键盘输入 n,编程计算 1!+2!+3!+……+n!

```
#include<stdio.h>
void main()
{
    int term,sum=0,i,j,n;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        term=1;
        for(j=1;j<=i;j++)
```

电脑无忧本网 WWW.515148.COM

```
        {
            term*=j;
        }
        sum+=term;
    }
    printf("1!+2!+3!+……+%d!=%ld\n",n,sum);
}
```

19 累加求和构成规律：

1）当累加项较为复杂或者前后项之间无关时，需要单独计算每个累加项

2）当累加项前项与后项之间有关时，根据前项计算后项

```
#include<stdio.h>
void main()
{
    int i,n;
    long term=1,sum=0;
    printf("Enter n:");
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        term=term*i;
        sum=sum+term;
    }
    printf("1!+2!+3!+……%d=%ld\n",n,sum);
}
```

20.goto 语句

1）功能：使程序无条件跳转到语句标号所标识的语句去执行,所跳过的语句不再执行

2）一般形式：

①向前跳转

```
        goto  语句标号;
        ……
语句标号:……  (前)
```

②向后跳转

```
语句标号:……(后)
        ……
        goto  语句标号;
```

3）应用：通常情况,goto 语句与 if 语句联合使用

```
        if(表达式) goto 语句标号;
        ……
语句标号:……
```

```
语句标号:……
        ……
        if(表达式) goto 语句标号;
```

21.break 语句

1）功能：①用于退出 switch 结构

②用于退出由 while,do-while 和 for 语句构成的循环体

2）原理：当执行循环体遇到 break 语句时,循环体将立即终止,从循环语句后的第一条语句开始继续执行

3）应用：break 语句通常与 if 联合使用,表明在任何条件下跳转到紧接循环语句后的第一条语句

22.continue 语句

1）功能：跳过 continue 后面尚未执行的语句,开始下一次循环,只结束本次循环,不终止整

甘肃老外本网 WWW.515148.COM

个循环

2）例题：

```
#include<stdio.h>
void main()
{
    int i,n;
    for(i=1;i<=5;i++)
    {
        printf("Please enter n:");
        scanf("%d",&n);
        if(n<0) continue;
        printf("n=%d\n",n);
    }
    printf("Program is over!\n");
}
```

23.函数 exit()

1) 标准库函数 exit()用于控制程序的流程,调用时,需要加头文件<stdlib.h>

2）一般形式:exit(code);

3）功能：终止整个程序的执行,强制返回操作系统,并将 int 型参数 code 的值传给调用进程(一般为操作系统),当 code 的值为 0 或宏常量 EXIT_FAILURE,表示程序出现某种错误后退出

24.goto,break,continue,exit()的比较

1）goto,break,continue,exit()都用于控制程序的流程,前三个是流程控制语言,exit()是 C 标准函数

1）功能：goto 语句可以向任意方向跳转,break 语句只限定流程跳转到循环语句之后的第一条语句,continue 语句结束本次循环,exit()直接终止所有程序

2）break,goto 语句和 exit()函数都可用于终止整个循环的执行,continue 不能终止整个循环

3）在嵌套循环下,break 语句和 continue 语句只对包含他们的最内层循环语句起作用,不能用 break 语句跳出多重循环,只能一层一层的跳出

4）使用 goto 语句的两种特定情形：

①快速跳出多重循环

②跳向共同的出口位置,进行退出前的错误处理工作

25.例题：韩信点兵:x%5==1&&x%6==5&&x%7==4&&x%11==10

①穷举法(循环条件自定义,不具实际意义)

②break 退出循环(循环条件省略,满足条件结束循环)

③exit(0)结束程序(循环条件省略,满足条件结束整个程序)

④使用标志变量(循环条件为标识变量为 0,最佳方法)

```
#include<stdio.h>
void main()
{
    int x;
    int find=0;
    for(x=1;!find;x++)
    {
```

WWW.515148.COM

```
        if(x%5==1&&x%6==5&&x%7==4&&x%11==10)
        {
            printf("x=%d\n",x);
            find=1;
        }
    }
}
```

26.类型溢出

1）原因：当运算的结果超出了类型所能表示的数的上界,导致进位到达了最前面的符号位或者更多进位的丢失,就会发生类型溢出

2）解决办法：采用取值范围更大的数据类型来定义变量

27.结构化程序设计的基本思想

1）采用顺序、选择和循环三种基本结构作为程序设计的基本单元,语法结构具有 4 个特性

①只有一个入口

②只有一个出口

③无不可达语句,即不存在永远执行不到的语句

④无死循环,即不存在永远都执行不完的循环

2）尽量避免使用 goto 语句,因为它破坏了结构化设计风格,并且容易带来错误的隐患

3）采用自顶向下、逐步求精的模块化程序设计方法进行结构化程序设计

甘肃专升本网 WWW.315148.COM